

Modern Datalog on the GPU

Yihao Sun
Syracuse University
Syracuse, USA
ysun67@syr.edu

Ahmedur Rahman Shovon
University of Illinois Chicago
Chicago, USA
ashov@uic.edu

Thomas Gilray
The University of Alabama at
Birmingham
Birmingham, USA
gilray@uab.edu

Kristopher Micinski
Syracuse University
Syracuse, USA
kkmicins@syr.edu

Sidharth Kumar
University of Illinois Chicago
Chicago, USA
sidharth@uic.edu

ABSTRACT

Modern Datalog engines (e.g., LogicBlox, Soufflé, ddlog, and RDFox) enable their users to write declarative queries which compute recursive deductions over extensional facts, leaving high-performance operationalization (query planning, semi-naïve evaluation, and parallelization) to the engine. Such engines form the backbone of modern high-throughput applications in static analysis, network monitoring, social-media mining, and business analytics. While previously-successful engines used compressed data structures (e.g., BDDs), modern in-memory engines typically employ iterated range-indexed nested-loop joins over explicitly-manifest relations.

In this paper, we present a methodology for implementing a modern in-memory Datalog engine on data center GPUs, allowing us to achieve significant (5–10×) gains compared to Soufflé (a modern CPU-based engine) on context-sensitive points-to analysis of httpd. We present GDLOG, a SIMD API which implements iterated relational algebra kernels over a novel range-indexed data structure we call the hash-indexed sorted array (HISA). HISA combines the algorithmic benefits of incremental range-indexed relations with the raw computation throughput of operations over dense data structures. We have currently implemented both CUDA and HIP variants of GDLOG, and have evaluated it on a range of data center GPUs including the H100, A100, MI250 and MI50. Our experiments show that GDLOG achieves performance competitive with modern SIMD hash tables and beats prior work by an order of magnitude in runtime while offering more favorable memory footprint.

KEYWORDS

Datalog, Recursive Queries, SIMD data structures, static analysis

ACM Reference Format:

Yihao Sun, Ahmedur Rahman Shovon, Thomas Gilray, Kristopher Micinski, and Sidharth Kumar. 2025. Modern Datalog on the GPU. In *Proceedings of (SIGMOD 2025)*. ACM, New York, NY, USA, 14 pages. <https://doi.org/XXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIGMOD 2025, 2025, Berlin, Germany

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXX.XXXXXXX>

1 INTRODUCTION

High-performance Datalog engines power state-of-the-art efforts in program analysis [6, 8, 13, 14], social-media mining [16, 38, 39], and business analytics [35]. Such engines enable a user to specify deductions (forming an *intensional* database, henceforth IDB) over extensionally manifest relations (the *extensional* database, henceforth EDB). For example, given an input relation $Edge(from, to)$, the following two rules compute $Edge$'s transitive closure ($REACH$):

$$\begin{aligned} Reach(from, to) &\leftarrow Edge(from, to). \\ Reach(from, to) &\leftarrow Edge(from, mid), Reach(mid, to). \end{aligned}$$

The first rule copies $Edge$ into $Reach$, and may be implemented via standard techniques (e.g., selection in SQL [11]). The second rule constitutes the bulk of the computation, and requires iteration to a fixed-point: at each iteration, the engine discovers possibly-new tuples in $Reach \bowtie Edge$ and merges them into $Reach$ until no new reachable edges are found. While traditional RDBMS systems do offer support for recursive queries in principle (e.g., SQL's common table expressions [7]), their usage in practice has been limited by their notoriously poor performance and lack of efficient query planning. These performance challenges stem from the substantively different concerns typically faced by large-scale deductive analytics tasks. Modern engines employ incrementalized (semi-naïve) evaluation, compiling queries into nested loop joins over in-memory, index-organized tables [43] represented via BTrees [22], tries [21], or a similar relation-backing data structure [36].

Unfortunately, best-in-class in-memory Datalogs hit scalability walls around 8–16 threads due to the challenges of working with locking, linked data structures in the shared-memory parallelism setting. For example, as we will soon see, when run at 32 threads on transitive closure, Soufflé (a state-of-the-art in-memory engine) spends 77.8% of its time in serialized tuple deduplication/insertion. By contrast, GPUs have thousands of threads: clearly CPU-based approaches will not suffice to enable scalable Datalog on the GPU. The crux of the matter is that Datalog's dense loop is range-indexed joins: unlike in SQL (where indices may not be known a-priori), Datalog engines ensure each relation is indexed so that it may always be efficiently range-queried rather than iterated.

Our engine, GDLOG, comes in the form of a SIMD API that provides parallel relational algebra kernels—essentially, a fully-join-planned Datalog program. We currently use Soufflé to perform join planning, and systematically translate its output to GDLOG's

kernels. GDLOG is backed by a novel SIMD data structure: the hash-indexed sorted array (HISA). We designed HISA to balance the concerns of modern Datalog on the GPU, enabling three critical tasks: (1) efficient range queries, necessary for joins, (2) lock-free deduplication, and (3) parallel insertion. Additionally, GDLOG provides abstractions for optimized evaluation of n -way loop-joins we call *temporarily-materialized* joins. These temporarily materialized joins offer a space-for-time trade-off that we found especially beneficial in systems with a high VRAM capacity.

We have evaluated GDLOG by using it to implement several deductive-analytic tasks and testing them at high scale versus the best-in-class CPU and GPU-based alternatives. Specifically, we have evaluated the performance of reachability, same generation, and context-sensitive program analysis on a diverse range of graphs, programs, and engines (comparing against cuDF, GPUJoin, and Soufflé). We observe much improved (5×) performance against state-of-the-art GPU-based joins due to efficient range querying, and also observe significantly reduced memory footprint. For example, in Section 5.4 we show cuDF OOMs on five of six graphs, while GPUJoin OOMs on two; GDLOG executes successfully for all six.

In sum, our contributions are as follows:

- The Hash-Indexed Sorted Array (HISA), a novel relation-backing data structure which provides range querying while leveraging the practical efficiencies of dense representations.
- GDLOG, a CUDA-based library for building high-throughput deductive analytics applications on the GPU; GDLOG uses HISA as its tuple representation. GDLOG also leverages two novel strategies apropos Datalog on the GPU: eager buffer management and temporarily-materialized k -ary joins.
- A thorough evaluation of data structure and optimization trade off used in GDLOG and a performance comparison between GDLOG and the SOTA CPU and GPU based Datalog systems.

We begin by surveying the design and implementation of modern CPU-based deductive database systems, particularly remarking upon their scalability bottlenecks. We next shift to discussing concerns regarding a relation-backing SIMD data structure for iterated joins on the GPU, before introducing our solution (the HISA) in Section 3. Section 4 then discusses how we leverage HISA to build GDLOG. Our evaluation is in Section 5, after which we conclude.

2 MODERN DATALOG ENGINES

Declarative languages for forward reasoning, such as Datalog, have yielded exciting results for a diverse set of data-heavy analytic applications such as deductive databases for machine learning or simulation analytics, graph mining, and static program analysis. There are a plethora of modern Datalog languages and engines, varying widely in their performance, maturity, portability, flexibility, and expressivity. For our discussion, modern Datalog implementations comprise two main categories:

- Stream-based (potentially distributed) [29, 30, 39], which attempt to use view-maintenance and similar approaches to scale incremental deductive reasoning to (e.g., internet-scale [28]) data streams, with work often being split between nodes on commodity or high-performance clusters.

- Batch, in-memory Datalog engines, which execute queries to a fixed point using high-performance iterated joins over relation-backing data structures in RAM [2, 13, 20, 37, 43].

The stream-based implementation methodology comprises an active research area with exciting results in both theory [9, 18] and practice [19]. In this paper, we restrict our focus to batch, in-memory Datalog engines (such as Soufflé). The restriction to batch computation allows such engines to achieve the best performance in terms of raw throughput for heavily recursive workloads such as program analysis or graph analytics [8, 38]. Such solvers are built on relation-backing data structures that expose parallel iterators, so that joins may be computed in a task-parallel manner. Indices are automatically generated so that joins may efficiently select relevant tuples instead of scanning and filtering. Unfortunately, multiple indices may be required in practice, increasing (linearly) time and space (for storage and updating). To ameliorate this, Soufflé uses a special index selection technique [43] that minimizes the number of indices provided relations are organized as curried/nested tries.

```
auto part = rel_delta_reach->partition();
// Parallel nested-loop join, pfor uses OpenMP
pfor(auto it = part.begin(); it < part.end(); it++) {
    for (const auto &env0 : *it) {
        // range query: edge indexed on its second column
        auto range = rel_edge->lowerUpperRange_10(...)
        for (const auto &env1 : range) {
            if (!rel_reach->contains(...) && ...)
                rel_new_reach->insert(...); } } }
// serial insertion from new to delta
for (const auto &env0 : *rel_new_reach) {
    // insert new tuple into full
    rel_reach->insert(...); }
```

Listing 1: C++ code generated by Soufflé (for REACH)

Listing 1 shows Soufflé’s compilation of transitive closure into semi-naïve evaluated relational algebra operations. The *Reach* relation is separated into three versions: new for tuples generated in the current iteration, *delta* for tuples from the previous iteration, and *full* for all tuples. Each of these relation versions is stored in a curried B-tree. The computation is only performed on the *delta* version to minimize redundant computation. Here the outer *Edge* relation’s *delta* version is partitioned based on its index and iterated in a parallel-for-loop (using OpenMP). Then, it leverages fast range queries on the inner *Reach* relation to perform a join operation. These results are then inserted serially into the *full* version of the output *Reach* relation (using locking insertions).

Scalability limits of shared-memory CPU systems. Despite Soufflé’s range partitioning for parallelizing joins, its tuple insertion remains serialized. Our benchmarks (Figure 1) show Soufflé’s limited scaling efficiency beyond 8 threads, with only a 7% improvement when increasing from 16 to 32 threads. A detailed analysis (Figure 2) reveals that tuple insertion into the full relation, taking up 77.8% of the runtime, is a major bottleneck in deduction-heavy workloads. Furthermore, even with potential parallelization, the curried B-tree structure’s locks and cache design hinder optimal parallelism with

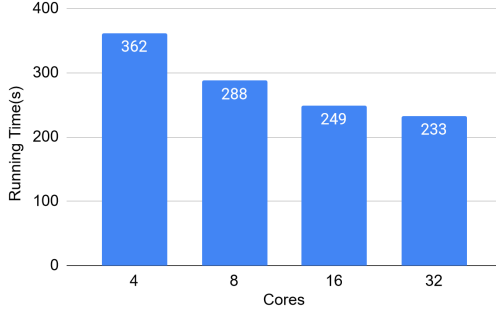


Figure 1: Scalability of *REACH* in Soufflé (com-dblp dataset).

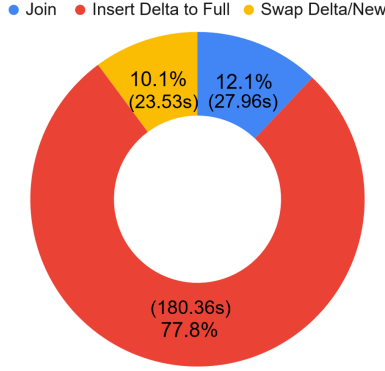


Figure 2: Runtime Analysis of Soufflé: Executing Reachability at 32 threads on the com-dblp dataset with 1.9B edges.

many threads. Besides memory insertion, data movement operations, including memory deallocation, also consume more time than relational algebra computations (12.1% compared to 10.1%).

Datalog on the GPU: Challenges and Opportunities. GPUs and similar hardware accelerators offer an attractive implementation candidate for Datalog backends due to their excellent performance in data-parallel, memory-intensive applications. Their programming model supports highly parallelized loops (with 10,000+ FP32 cores on modern GPUs), making them an apparent match for nested loop joins. Crucially, GPUs provide extremely high throughput: the recent AMD MI300 can transfer 8192 bits in a single clock cycle. Our goal is to use this throughput to sidestep the memory bandwidth limitations faced by CPU-based Datalog engines.

There are several challenges in implementing state-of-the-art Datalog engines on the GPU. First, we need a GPU-based representation of relations (sets of tuples), which are typically implemented via linked data structures on the CPU. Second, the massive number of parallel threads dictates that lock freedom and communication avoidance be particularly relevant concerns. GDLOG uses lock-free data structures by restructuring performance-critical computations (such as sort, join, and merge) into a two-phase process. Since GPUs lack dynamic allocation, our initial pass involves allocating a sizable buffer in the GPU global memory to accommodate the results based on dynamic relation sizes. Next, individual kernel threads perform calculations for their respective segments of the buffer,

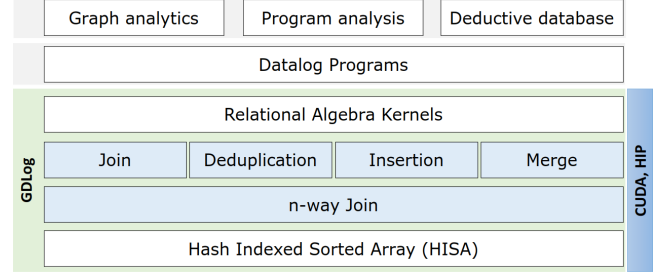


Figure 3: High level block diagram of GDLOG.

determined by their unique thread IDs, and proceed to write their corresponding results into the allocated space.

Along with the chief algorithmic concerns, effectively using a modern GPU involves other challenges such as efficient deduplication of parallel results, minimizing CPU-GPU communication, and optimizing memory coalescing for peak performance. Overcome these obstacles and ensuring performance portability across diverse GPU architectures are essential steps towards creating a high-performance GPU Datalog.

GPU Datalog: Prior Work. There exist several relevant threads of related work in GPU Datalog. However, state-of-the-art work in GPU Datalog often compromise on the expressiveness of full Datalog to achieve their results. For example, Red Fox omits semi-naïve evaluation and recursive queries[46], essential for modern Datalogs. GPUDatalog employs more similar to the GPU RDBMS GDB [17], successfully executing such as *REACH* and *SG* by constructing Cache Sensitive Search (CSS) Trees [32] before each join iteration. Unfortunately, data organization and storage limits inhibit GPU-Datalog’s ability to efficiently deduplicate data, and GPUDatalog also eschews semi-naïve evaluation.

More recently was the work of GPUJoin by Shovon et al [42]. GPUJoin’s initial implementation [41] utilized NVIDIA’s RAPIDS project cuDF [34], but struggled with memory-intensive applications due to inefficient memory and buffer management. Its subsequent version improved performance by using an open-addressing hash table for tuple storage, achieving results closer to that of Soufflé’s interpreted mode [42]. However, GPUJoin doesn’t significantly outperform CPU-based systems—even on modern data center GPUs. Additionally, GPUJoin’s data structures are application-specific, significantly impeding expressivity by prohibiting relations with arity greater than two.

Our approach. Despite the limitations of existing GPU Datalog prototypes, the significant benefits of massive parallelism and the rapid advancement of GPU hardware continue to motivate us in further developing performance portable GPU-based Datalog engines targeting modern heterogeneous computing systems. Thus, we have developed GDLOG, a systematic and comprehensive GPU Datalog solution compatible with various data center GPUs., as illustrated in Figure 3. GDLOG is capable of executing Datalog-based applications by translating modern Datalog engine join plans, such as those used by Soufflé, into relational algebra operations. It introduces the Hash Indexed Sorted Array (HISA), a data structure optimized for GPUs that supports all the parallel computation phases

crucial for semi-naïve evaluation. This approach sets GDLOG apart from most existing prototypes, which typically compromise on semi-naïve evaluation or essential Datalog language features like multi-ways join and support for relations of multiple arities. In the subsequent sections, we will present the following key features of our implementation:

- Section 3 introduces our memory-optimized, hash-based indexed data structure (HISA) designed for GPU efficiency.
- Section 4 discusses how GDLOG leverages HISA to implement a scalable and efficient semi-naïve evaluation.
 - Section 4.1 describes an efficient implementation of n-way iterative joins on new data structure, which has not yet been achieved in existing GPU Datalog research.
 - Section 4.2 outlines a trade-off in buffer management in the 2-phase GPU parallel computation.
- In Section 4.3, we explore GDLOG’s adaptability across various GPU models.

3 THE HASH-INDEXED SORTED ARRAY

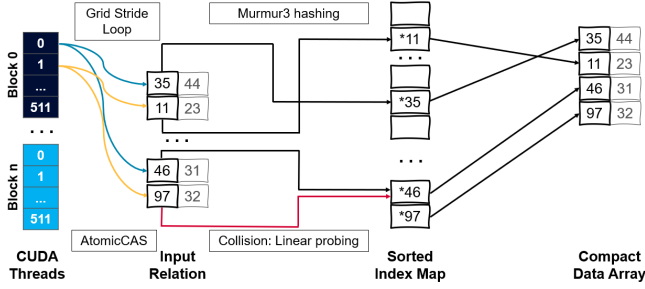


Figure 4: Tuple storage layout of relation $edge(f, t)$. The sorted index map is created based on the join column of the input relation pointing to the index in the compact data array.

Deduction-heavy workloads on GPUs demand efficient data structures that enable fast indexing, sorting, and range queries. This is why we introduce the Hash-Indexed Sorted Array (HISA), a hybrid SIMD data structure designed for optimal execution of deduction-heavy workloads on the GPU. The primary aim of HISA is to seamlessly integrate the algorithmic efficiencies inherent to indexing with the exceptional performance offered by sorted arrays. It is inspired by the compact and serializable nature of *HashGraph* [15] while tailoring it for efficient range queries, a crucial aspect for deduction tasks. At a high level, HISA establishes mappings between a compact data array stored in GPU memory and a sorted index map that enables fast lookups and range queries.

Figure 4 showcases the key steps of HISA: a mapping between a compressed data array and a sorted index map. This mapping is established using a grid stride loop executed by the kernel (CUDA/HIP) threads. The process involves sorting the join column of the input relation and linking it with the compact data array. The first step involves building the sorted index map. The chosen join key determines the order, resulting in keys that are 35, 11, 46, and 97 in the illustration. The sorted index map creates an open-addressing-based hash table with a low load factor and inserts the key pointers in lexicographic order. Collisions, arising from duplicate keys, are handled via linear probing which is suitable for GPU

architecture. For the hashing algorithm, we used *Murmur3* hash function [23] and invoked CUDA atomic operation for parallel hash table construction. Finally, the index map thus contains the sorted join key index pointer (*11, *35, *46, *97), which enables a faster range query and efficient deduplication process. Each key of the sorted index map is a pointer to the tuples in the GPU memory, forming the compact scalar data array. This connection between sorted keys and their corresponding data allows for rapid range queries and efficient deduplication, making HISA an ideal choice for deduction-heavy GPU workloads.

Algorithm 1 Hash table construction algorithm

```

1: for each tuple  $T$  in input relation perform parallel do
2:   Calculate hash value  $H$  for tuple  $T$ 
3:   Calculate index  $I$  as  $H \pmod{\text{index map size}}$ 
4:   while true do
5:     Check key  $K$  and value  $V$  at index  $I$  in hash table
6:     if  $K$  is empty or  $K$  equals  $H$  then
7:       while true do
8:         if  $V < I$  and tuple at  $V$  not equal to  $T$  then
9:           Collision detected, increment  $I$ 
10:          Continue outer loop
11:        end if
12:        if  $V$  greater than tuple index then
13:          Swap  $V$  with tuple index of  $T$ 
14:        end if
15:        break inner loop
16:      end while
17:      Insert  $(H, \text{tuple pointer of } T)$  into hash table at  $I$ 
18:      break outer loop
19:    else
20:      Increment  $I$ 
21:      Continue outer loop
22:    end if
23:  end while
24: end for

```

3.1 Deduplication and Parallel Insertion

In Datalog, deduplication of tuples within fixpoint iterations is a critical operation. Prior work has demonstrated that performing deduplication of intermediate results within each iteration affects the overall performance of recursive or iterative operations in Datalog evaluation [41, 42].

One crucial reason for maintaining values sorted in our indexing map is to optimize this process for GPU execution. The GPU’s exceptional parallelism lends itself to scan sorted arrays efficiently, enabling the rapid identification and removal of duplicates. In contrast, in a conventional multi-value hashtable, ensuring the uniqueness of values necessitates probing and comparing with all tuples within the same hash bucket, which can result in unnecessary redundant comparing within the bucket when inserting a new value. This approach is less efficient than the combination of fast radix sorting and unique operation, which takes full advantage of the GPU’s parallelism. Additionally, sorting for deduplication simplifies the data structure merge operation, as it boils down to merging sorted

values in the indexing map. This merging process is also highly GPU-friendly, achieved by coarsely partitioning the indexed map into several sorted chunks in GPU memory and merging them with fast parallel primitives. Overall, this sorted merge design harnesses the strengths of the GPU for massively parallel sorting, scanning, and merging and significantly enhances the efficiency of the deduplication operations in the fixed-point iterations.

3.2 Efficient range querying and serialization

Inductive queries in Datalog, which entail iterative fast range queries on indexed columns, benefit from hash-based indexing. This method, offering $O(1)$ range fetching time, is significantly advantageous over sort-merge joins. Storing only the join column data in the hash table enables us to achieve efficient insertion and retrieval operations based on key values. To address the serialization requirements during operations such as recursive joins on the GPU, and merging delta data into the full dataset, HISA’s compact sorted data array become useful. Serialization can be efficiently executed through parallel iteration over this structured array. Moreover, the hash table only contains the columns required for joins, not the full tuples. This optimization significantly reduces memory usage while still ensuring fast join performance. The hash table construction is shown in Algorithm 1. The bulk of the data is stored in a scalar array tailored for GPU architecture. Simultaneously, for inner relations that require fast range queries, we employ a hashmap designed to store only the indexed columns and the address of the smallest tuple pertaining to that indexed column. This approach optimizes space utilization while maintaining the efficiency of range queries, a key consideration in the context of Datalog and similar declarative languages. While *HashGraph* [15] demonstrates the effectiveness of combining a hashmap with a compact array-style data structure on GPU, it does not directly align with the specific demands of Datalog. Notably, it lacks support for handling multi-values and deduplication, which are essential aspects of Datalog queries. HISA addresses these limitations by introducing a sorted hash-indexed table mapped with the compact data array. This innovation allows us to utilize mainstream GPU merge and sort algorithms. By combining the sorted index map with the compact data array in this manner, we obtain a high-performance data structure tailored to the needs of recursive joins on the GPU. HISA’s design aims to provide the best of both worlds - efficient indexing and high-performance data manipulation - to support Datalog workloads on GPU architectures.

3.3 Implementation of HISA

The CUDA interface for our HISA data structure is presented in Listing 2. In HISA, all data is stored in C-style structs named “GHashRelContainer”. This struct combines an open-addressing hashtable, referred to as “index_map” in the code, with a compact array for sorted data, designated as “data_raw” in the code. The entire structure is stored in unified GPU memory. All operations on HISA are conducted through four distinct GPU kernel functions. The “load_relation_container” function initially employs a GPU sorting algorithm to sort all the tuples and subsequently stores these sorted tuples in GPU memory. The kernel function “create_index” then constructs the hashtable based on the loaded raw data within a

```
using column_type=rmm::device_vector<u32>;
struct GHashRelContainer {
    // open addressing hashmap for indexing
    OpenAddressMap *index_map = nullptr;
    tuple_size_t index_map_size = 0;
    float index_map_load_factor;
    // flatten tuple data
    device_vector<tuple_size_t> sorted_scalar;
    // will be init to arity size
    column_type *data_raw = nullptr;
    tuple_size_t tuple_counts = 0;
    int arity; bool tmp_flag = false;
};

__global__ void load_relation_container(
    GHashRelContainer *target, int arity, column_type &data,
    tuple_size_t data_size, tuple_size_t index_column_size,
    float index_map_load_factor);

__global__ void create_index(
    GHashRelContainer *target, tuple_indexed_less cmp);

__global__ void get_join_result_size(
    OpenAddressMap *inner, column_type &outer,
    int join_column_counts, tuple_size_t *join_result_size);

__global__ void get_join_result(
    OpenAddressMap *inner, column_type &outer, int arity,
    int join_column_counts, column_type *output_raw_data);
```

Listing 2: HISA data structure’s CUDA C API interface

HISA object. It’s worth noting that HISA is a specialized data structure designed specifically for deductive reasoning, and as such, it does not offer a general method for fetching tuples, as a typical hashtable would. Instead, HISA provides only two kernel functions: “get_join_result” and “get_join_result_size” for implementing joins.

In CPU-based Datalog engines, joins can be readily implemented by iterating over one relation and comparing each tuple in the iterator with the other relation. However, this iterator-style approach doesn’t align with the philosophy of SIMD processors such as GPUs. In GPU-based data structures, all joined column comparisons must be bundled together as a bulk operation.

In traditional GPU-based hash data structures, such as the static hashmap in NVIDIA’s *cuCollection* [10], a bulk retrieve function and a parallel serialization function are typically defined. Hash table-based deductive engines like Datalog queries in cuDF, implement parallel joins by serializing one join candidate’s (the outer relation) hashtable into an array and utilizing the GPU hashtable’s bulk API to retrieve all tuples that share the same join columns. A GPU kernel function is then used to project these matched tuples into join result tuples. However, when using HISA, joins are implemented by directly invoking the “get_join_result_size” and “get_join_result” CUDA kernels, which take the inner relation’s hashtable and the outer relation’s sorted array as input. The serialization function becomes unnecessary because all of our tuples are stored in a compact array that is serialized by default. Our compact data representation increases data locality by facilitating coalesced memory accesses—crucial in extracting performance from GPUs. Similar to other implementations of hash tables, in our join result kernel HISA also uses bulk data retrieval to allow fast parallel-range queries.

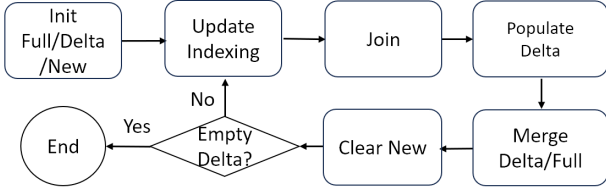


Figure 5: Workflow of semi-naïve evaluation in GDLOG.

Managing output buffer size is another challenge that one must deal with while performing joins. This is because joins are irregular operations, where the output size is not known in advance. Therefore, it becomes challenging to determine how much GPU memory one must allocate to store the result before actually executing the join operation. To address this challenge, we employ a two-step approach. First, we perform the join operation once to compute the size of the result (using “get_join_result_size”), and then allocate GPU memory for the result buffer based on the computed size. After the memory is allocated, we repeat the join operation (using “get_join_result”) to actually fetch and store the join result in the allocated buffer. This approach ensures that we don’t need to pre-allocate an oversized buffer to store the join result.

4 USING HISA FOR DATALOG ON THE GPU

In this section, we detail three challenges that remain to use HISA as the backbone for GDLOG. The first is handling semi-naïve evaluation. Semi-naïve evaluation is essential for algorithmic complexity of modern Datalog solvers, but ignored by much prior work on GPU Datalog. Next, we discuss a novel optimization we provide which allows us to significantly accelerate n-way joins: temporary materialization. Finally, we discuss the memory-for-time trade-offs employed in GDLOG’s buffer management strategy.

All modern Datalogs utilize semi-naïve evaluation, wherein a *frontier* of tuples are tracked. Semi-naïve evaluation avoids redundant work by recognizing that only newly-added facts will potentially trigger the materialization of new facts. To implement semi-naïve evaluation, a relation is split (e.g., as an intermediate stage of a Datalog compiler) into three versions: “new” (containing newly generated tuples in each iteration), “delta” (holding unique tuples added in the last iteration), and “full” (comprising all tuples discovered in all iterations). By performing the join operation solely on the “delta” relation, a substantial amount of unnecessary computation is eliminated. Every fixed point iteration of semi-naïve evaluation entails a series of steps: (a) maintaining indices on each joined relation, (b) executing the relational algebra kernel on *delta*, and (c) removing duplicates in the *new* relation and storing it as the *delta* (to be used as the input to the next iteration), and (d) merging the *delta* of every relation with *full* and emptying *new*. The end-to-end pipeline is sketched in Figure 5. We explain each of these steps in the following sections.

Updating Relations. Deductive queries, require iterated executions to a fixed point, demanding extensive range queries on inner relations. These range queries can be sped up using indices built on the relations. The first phase of the evaluation workflow is therefore

to update the indices for each relation. We adopt a strategy similar to Soufflé. We create a HISA index for every relation involved in a join operation. For instance, in the second rule of the *REACH* query (from Section 1), where the “Edge” relation is joined based on the second column, a copy of “Edge” is generated where the relation is indexed on the second column (the join column). This copy is denoted as $Edge_{2,1}$ – denoting that the second column is used for index computation. All subsequent recursive computation of the second rule in the *REACH* query operates on this copy. Furthermore, when a relation gains new tuples after merging delta and full relations, these merged tuples are synchronized with the relation’s indexed copy, ensuring that queries in the next relation can efficiently perform range queries.

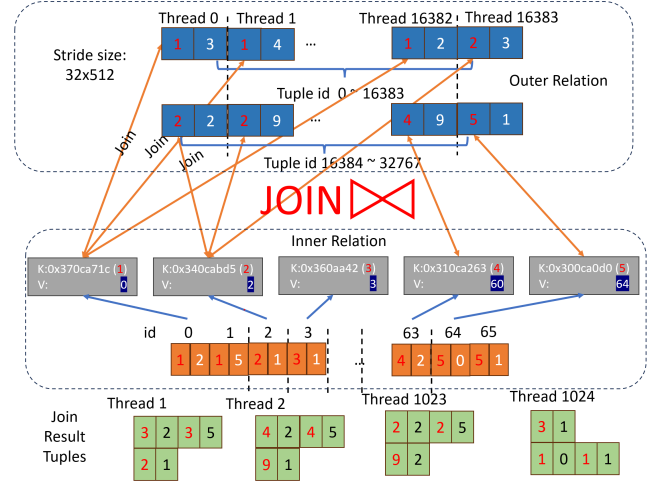


Figure 6: Example of a join between relations *Edge* (inner) and *SG* (outer) using a stride size of 16,383 CUDA threads

Join. The second step in the fixpoint computation involves the execution of join operations in GDLOG. Leveraging our hybrid data structure, these join processes are designed to harness the benefits of both hash and sorted joins, making them well-suited for recursive query scenarios on GPUs. We use a join operation from the second rule of the *REACH* query to illustrate this process (see Figure 6).

In the upper section of the figure, the outer relation (in this case, the delta relation of the *Reach* relation) is serialized and divided into chunks or arrays known as “strides”. The size of each stride corresponds to the total number of threads employed for concurrent execution during the join process. Users can configure the stride size based on their GPU hardware specifications, with a recommended size being 32 times the number of stream processors. In this example, we use a stride size of 16,394.

Once the data is partitioned, each worker thread can readily access tuples within the stride array according to its unique thread ID number. Thanks to GDLOG’s compact sorted array storage of relation tuples, the process of serializing and partitioning a relation becomes streamlined. This stands in contrast to a pure hash join, where efficiently iterating over the hash table and optimizing the distribution of join operations poses a significant challenge.

The lower section of the figure demonstrates how each thread queries the inner relation and generates results. As discussed in

Section 3, every relation contains a hash table storing key-value mappings based on the hash values of unique joined columns. These mappings indicate the offset of the smallest tuples that share the same indexing columns within the sorted array containing all relation tuples. Consequently, during the join process, each thread calculates the hash value of the joined columns, queries the inner relation’s indexing hash table, and locates the smallest tuple sharing the same joined columns. Since all tuples are sorted, a linear scan of the relation’s internal array starting from the smallest tuple offset facilitates the join. For instance, in the figure, thread 0 operates on the outer relation tuple $Reach(1, 3)$. It computes the hash value of the indexed column (in this case, the first column), which results in the value $0x370ca71c$. It then queries the indexing hash table of inner relation $Edge_{2,1}$ with this value and discovers that the smallest tuple with 1 as its first column is located at offset 0 in the internal array of relation $Edge_{2,1}$. Subsequently, a scan of the sorted data array facilitates the join of the tuple $Edge_{2,1}(1, 2)$ and $Edge_{2,1}(1, 5)$, generating join result tuples $Reach(3, 2)$ and $Reach(3, 5)$. It’s important to note that while we can obtain the join result at this point, due to GPU limitations, on-the-fly buffer allocation is not possible. The standard practice, also adopted by other works such as RateupDB and GPUJoin, is to repeat the above join operation twice. The first pass is used to compute the size of the join result, followed by writing the join result into the result buffer during the second round of join.

Before storing the join results in the new version of the output relation, an important additional step is the deduplication of join results in each iteration, so that no repetitive computation is done in the next iteration. GDLOG accomplishes this by utilizing sorting based on the lexicographic order of tuples, followed by a scan to eliminate sequential duplicates. Both of these operations are parallelized by the GPU and are readily implemented using functions from NVIDIA’s Thrust library.

Populating delta. The next step in the fixpoint loop involves populating the delta relation, which will be used in the join phase of the next iteration. This step is realized by examining the current full relation and removing the duplicated tuple already present in the new relation. GDLOG accomplishes this using a relational algebra operator known as “set difference” applied to the new and full relations. It’s worth noting that some prior work, such as GPUJoin, fuses this step with the merging step by directly merging the non-duplicated delta relation and then deduplicating the merged full relation. This fusion approach is efficient when the size of the full relation is small. However, deduplicating the full relation requires a full scan of all the tuples in the full relation, which becomes highly expensive as the size of the full relation grows. Therefore, GDLOG takes a different approach by separating the delta relation population into a distinct phase.

Merging Full with Delta and Clearing New. The final step before fixpoint checking involves merging all the tuples within the delta relation into the full relation, followed by the removal of all tuples within the new relation. As we’ve previously discussed in the background and motivation section, this particular phase is inherently serial and has the potential to become a bottleneck when dealing with larger query sizes in CPU-based Datalog engines. In GDLOG we can take advantage of GPU operation over our data structure’s

internal compact array to parallelize this process. Efficient parallelization is achieved by employing a strategy that entails the coarse yet swift partitioning of both the sorted array of full and delta relations into several small, sorted tiles, each of which can fit neatly into a GPU’s execution wraps. This transformation of the merging process effectively converts it into a divide-and-conquer problem and, as a result, makes it amenable to seamless acceleration by SIMD hardware.

4.1 Temporarily-Materialized N-way Joins

While simple Datalog queries such as reachability use only binary joins, nontrivial queries use n-way joins. An example is the Same Generation query (SG), which ascertains which nodes in an arbitrary graph share an identical topological order:

$$\begin{aligned} SG(x, y) &\leftarrow Edge(p, x), Edge(p, y), x \neq y. \\ SG(x, y) &\leftarrow Edge(a, b), SG(a, b), Edge(b, y). \end{aligned}$$

The second rule of SG represents an inductive case that necessitates joining SG with $Edge$ twice within the same iteration. When employing semi-naïve evaluation, its join plan may be expressed via the following relational algebra operation:

$$SG_{new} = SG_{delta} \bowtie Edge_{full} \bowtie Edge_{full}$$

Various mechanisms exist for evaluating n-way joins; while special-purpose joins (such Leapfrog Triejoin [44]) can achieve worst-case optimal bounds, they do so by imposing representation restrictions. Modern engines tend to favor nested-loop joins [20], in which case n-way joins are ordered into sequences of binary joins. In the case of SG, there are two distinct methods for dividing the join into sub-joins and allocating them across worker threads. For example, one strategy involves partitioning the entire join based on tuples in the outermost relation (*i.e.*, tuples in SG_{delta}).

Multi-core CPUs operate on a Many Instruction Many Data (MIMD) architecture, allowing each thread to execute different instructions concurrently without experiencing idle time caused by conditional branching in the program. However, this workload division approach doesn’t align with the hardware nature of GPUs. GPUs utilize a Single Instruction Many Data (SIMD) architecture, where threads within the same GPU warp can only execute the same instruction at any given time. Consequently, conditional branching in the program can lead to some threads idling.

To understand the impact of SIMD execution on n-way joins, consider the example in Figure 7. Thread 0 is assigned to compute the join result for the tuple $SG(1, 3)$. During the first join with $Edge_{full}$, it generates numerous temporary results. Unfortunately, thread 30 and thread 31 lie within the same GPU warp as thread 0 do not find any matching tuples. In the subsequent join between the first set of join results and $Edge_{full}$, thread 0 must continue the computation, while threads 30 and 31, even though they have no work to do, cannot proceed to the next outermost tuple. They remain idle until thread 0 completes the second join and returns.

Another approach to dividing the workload among threads involves materializing each sub-join. We leverage this observation by using a “temporary materialized” buffer to accelerate n-way SIMD joins. In the case of SG, the second rule can be seen as two

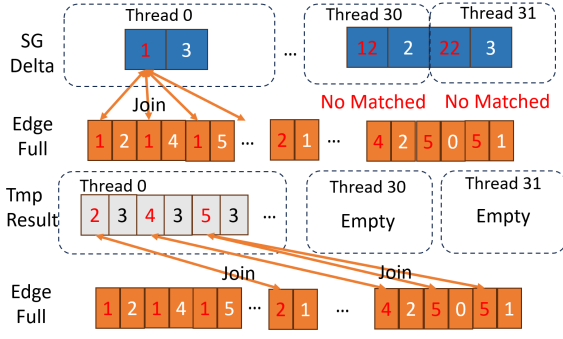


Figure 7: Thread waiting in nested non-materialized join due to workload imbalance within the same GPU warp

sequential binary joins:

$$\begin{aligned} \text{Tmp}(b, x) &\leftarrow \text{Edge}(a, b), \text{SG}(a, b). \\ \text{SG}(x, y) &\leftarrow \text{Edge}(b, y), \text{Tmp}(b, x). \end{aligned}$$

In relational algebra, an explicit join plan may be written as:

$$\begin{aligned} \text{Tmp} &= \text{SG}_{\text{delta}} \bowtie \text{Edge}_{\text{full}} \\ \text{SG}_{\text{new}} &= \text{Tmp} \bowtie \text{Edge}_{\text{full}} \end{aligned}$$

In CPU-based systems, this split doesn't make a significant impact on performance. However, in GPU-based systems, having two sequential joins scheduled into two separate executions of GPU kernel functions is notably different from the first method, where only one GPU kernel function executes.

For the first join's workload division, it's still based on tuples in SG_{delta} . In contrast, the workload of the second join is divided based on the Tmp relation tuples. In the previous example, temporary join result tuples like $\text{Tmp}(4, 3)$ and $\text{Tmp}(5, 3)$ are assigned to other idle threads, such as thread 30 and thread 31, rather than all being processed by the busy thread 0. In this way, all threads in the same GPU wrap will have a similar workload, eliminating idle threads.

The main drawback to temporarily-materialized joins is the extra space overhead required to store the temporary relations. However, unlike in shared-nothing approaches (such as BPRA [24])—wherein intermediate results must be persistently materialized in space—our approach purges temporarily-materialized tuples once the associated join completes. Additionally, unlike normal relations, temporarily-materialized results do not require multiple versions, sorting, or deduplication.

4.2 Trade-off: Eager Buffer Management

While the GPU merge operation used to merge full and delta exhibits excellent time performance, it is not without its costs. An intermediate step in the GPU merge algorithm necessitates the loading of two merge candidates into a sequentially shared memory buffer, the size of which must be at least equal to the sum of the candidates' memory sizes. As the recursive query progresses, the full relation size continually grows, subsequently increasing the memory requirement for this buffer in each iteration.

Allocating a substantial memory buffer during each iteration is not only space-inefficient but also time-consuming. As demonstrated in our experiments in the evaluation section (Section 5.3),

without optimization, buffer allocation can consume over 65% of the total execution time in certain test cases. To address this issue, GDLOG employs an optimized buffer management strategy. We named it as Eager Buffer Management (EBM).

Algorithm 2 Allocate buffer for merging delta and full.

Require: allocatedBufferSize, deltaSize, fullSize

Ensure: currentMergeBuffer

```

1: if allocatedBufferSize < deltaSize + fullSize then
2:   return currentMergeBuffer
3: end if
4:  $K \leftarrow \alpha$  ▷ We set  $\alpha = 5$ 
5:  $\text{freeMemory} \leftarrow \text{getFreeGPUMemory}()$ 
6: while fullSize + deltaSize  $\times K$  < freeMemory do
7:    $K \leftarrow K - 1$ 
8:   if  $K = 0$  then
9:     throw error("not enough memory")
10:  end if
11: end while
12:  $\text{buffer} \leftarrow \text{allocate}(\text{fullSize} + \text{deltaSize} \times K)$ 
13: return buffer

```

The central concept of EBM is to pre-allocate a buffer larger than necessary during the initial buffer allocation for merging. This larger buffer can then be reused across multiple iterations, saving the time required for buffer allocation in subsequent iterations. To reuse buffers, GDlog also needs to maintain these larger-than-necessary buffers throughout the execution of the query, which can lead to additional GPU memory consumption. Our experiments have shown that performance improvement (in time) comes at a 1.35× space overhead. This additional memory used for the larger buffer allocation becomes a space-for-time trade-off to optimize buffer allocation efficiency in later iterations.

We introduce the detail of our memory management strategy in Algorithm 2. This algorithm is executed before merging the full and delta relations in each iteration to prepare the required GPU merge buffer. The algorithm begins by checking whether the existing buffer size ('allocatedBufferSize') is sufficient to accommodate the delta tuples generated in the current iteration ('deltaSize') in addition to the tuples in the full relation ('fullSize'). If this condition is met, it means that the GPU merge operation can reuse the buffer allocated in the previous iteration ('currentMergeBuffer').

If the existing buffer is not large enough, rather than creating a buffer with a total size equal to the sum of the full and delta relations, GDLOG suggests allocating a new buffer with a size of $\text{fullSize} + \text{deltaSize} \times K$, where 'K' is initially set to 5. The algorithm then checks whether the proposed buffer size exceeds the available GPU memory size ('freeMemory'). If the proposed size is too large, the algorithm gradually reduces 'K' until it can fit within the remaining GPU memory. If 'K' is reduced to 0, it indicates that there is insufficient GPU memory to complete the join operation, leading to an "Out of Memory" (OOM) exception being thrown. This proactive approach optimizes memory allocation for efficient semi-naïve evaluation.

In real-world query scenarios, it is an uncommon occurrence for the evaluation process to exhibit abrupt convergence, especially

when the delta relation contains numerous tuples. Conversely, the size of the delta relation tends to gradually decrease in the last few iterations toward the final fixpoint. During these tail iterations, creating buffers becomes a costly operation because the full relation contains a considerable number of tuples. However, the merge step, where the real computation occurs, takes almost no time because the delta tuples used in merging is small.

Eager Buffer Management effectively reduces the buffer allocation overhead during tail iterations, introducing only a small amount of additional memory overhead. This makes it a particularly cost-effective trade-off for queries characterized by long tail behavior, a common occurrence in network analysis tasks. However, for queries with a short tail, such as the Same Generation query mentioned in the previous section, it's advisable to disable this optimization to avoid unnecessary memory overhead. By customizing this strategy based on the query's specific characteristics, GDLOG can optimize its performance for various scenarios.

4.3 Porting GDLOG to HIP

To increase the performance portability of GDLOG, we translated it to a Heterogeneous-Compute Interface for Portability (HIP)[1] based engine (GDLOG-HIP hereafter) with identical function signature of the former Datalog engine. Thus, we have the ability to run the analytical programs in both NVIDIA and AMD GPUs seamlessly. In both CUDA and HIP, memory allocation plays a crucial role in enabling efficient parallel processing on GPUs. CUDA, developed by NVIDIA, and HIP, an open-source alternative primarily supported by AMD, share similarities in their approach to memory management but also exhibit notable differences. CUDA utilizes a specific set of memory allocation functions to allocate and deallocate memory on the GPU device, enabling data transfers between the CPU and GPU. CUDA's memory allocation is tightly integrated with its programming model, offering developers a streamlined experience when working with NVIDIA GPUs.

In GDLOG with a CUDA backend, we utilize NVIDIA's RAPIDS Memory Manager (RMM) library [33] to enhance memory allocation and deallocation performance. However, an equivalent of RMM for the HIP is not available. As a result, our implementation on the HIP backend relies on the standard memory allocation functions provided by the HIP API, which may potentially lead to slower memory operations. Despite this, HIP's cross-compatibility with various GPU architectures is a significant advantage, enabling the easy adaptation of GDLOG to upcoming high-performance computing platforms, such as the exascale supercomputers Frontier and Aurora [3, 31], which may not rely on NVIDIA hardware and do not have CUDA support.

5 EVALUATION

In this section, we conduct a comprehensive performance evaluation of GDLOG, demonstrating the impact of our optimization techniques. Our assessment begins by analyzing how eager buffer management influences both query execution time and system memory usage. We then compare the performance of GDLOG against existing Datalog engines, utilizing well-established Datalog queries and real-world datasets. Due to the limited availability of mature

GPU-based Datalog engines and the inaccessibility of GPUDataLog [27] in both source and binary forms, we also include a state-of-the-art CPU-based system in our comparison. Specifically, we benchmark against Soufflé, renowned for being one of the fastest Datalog engines. Although claims of a recent system DCDatalog [47] surpassing Soufflé in speed, their artifacts are not accessible for evaluation. Finally, we demonstrate the practicality of GDLOG for program analysis, showing significant performance advantages over current CPU-based solutions.

5.1 Environment Setup

Before diving into the details of our evaluation, we provide a brief overview of our experimental environment settings:

GDLOG and Other GPU-Based Systems. All our CUDA-related code runs on a server equipped with a 64 core AMD EPYC 7713 CPU, backed by 128GB of DRAM, and features one NVIDIA H100 80GB PCIe GPU (driver version 525.78). GDLOG is compiled with CUDA version 12.0. For cuDF, we employ version 23.10, the latest version available at the time of our evaluation. The code for Datalog queries in cuDF and GPUJoin is sourced from its public GitHub repository [40], employing the same compiler toolkit as GDLOG.

Our GDLOG-HIP implementation is tested on 2 machines. One is a Supermicro AS-4124GQ-TNMI server with dual AMD EPYC 7713 processors together with four 64GB HBM2e AMD Instinct MI250 cards, supported by 512GB DDR4 RAM. Another is a Gigabyte G482-Z51 server, integrates dual AMD EPYC 7742 and four AMD MI50 GPUs with 32GB VRAM and a 256GB DDR4 RAM. In our benchmark only one GPU in each system is used. ROCm 5.7.0, CMake 3.27.2, CUDA 11.2.2, and GCC 13.1.0 are used to build GDLOG-HIP.

CPU Based System: Soufflé. Our Soufflé experiments are conducted on a node within Polaris supercomputer [4] situated at the Argonne Leadership Computing Facility (ANLF). This supercomputer's computation nodes boasts a top-tier multi-core CPU, the AMD EPYC 7543P, featuring 32 cores clocked at 2.8GHz and supported by 512GB of DRAM. Soufflé is configured using the CMake release build mode and compiled with GCC 10. All Datalog queries are compiled into C++ code using compilation flag `-j 32`, enabling the utilization of all CPU cores on the test machine. We found this delivered the best runtime of any thread-count.

Because the focus of our experiment is to evaluate the performance of the deductive engine itself, we want to exclude any potential impact of join planning on performance. Thus, we have Soufflé emit its join plan via the flag `-show=transformed-ram`. We then ensure that all tools used in our experiments employ the same join plan as Soufflé by manually replicating Soufflé's plan in our implementations using GDLOG, cuDF, and GPUJoin.

5.2 Test Programs and Datasets

We evaluate three distinct queries: reachability (*REACH*), same generation (*SG*), and Context-Sensitive Program Analysis (*CSPA*). We include *REACH* as a common baseline which stresses iterated binary joins, without any opportunity for task-level parallelism across rules. We include *SG* and *CSPA* as queries which require more intelligence in join planning and stresses support for n-way joins. Recall the code for *SG* was introduced in Section 4.1.

Table 1: Comparing runtime and memory usage of *REACH* in GDLog with and without eager buffer management.

Dataset	Iterations		Query Time (s)		Memory(GB)	
	Total	Tail	Normal	Eager	Normal	Eager
usroads	606	/	52.42	17.53	20.35	26.84
vsp_finan	520	491	59.08	21.91	20.22	28.26
fe_ocean	247	90	47.19	23.36	37.97	50.43
com-dblp	31	18	17.83	14.30	43.24	60.18
Gnutella31	31	17	4.80	3.76	20.22	28.26

We conducted our experiments using ten real-world graphs for both the *REACH* and *SG* queries. These datasets were carefully chosen from the Stanford Network Analysis Project network dataset, SuiteSparse, and road network datasets [12, 25, 26]. These datasets vary in size and originate from diverse source applications. For *CSPA*, we used input data provided by the authors of RecStep [13].

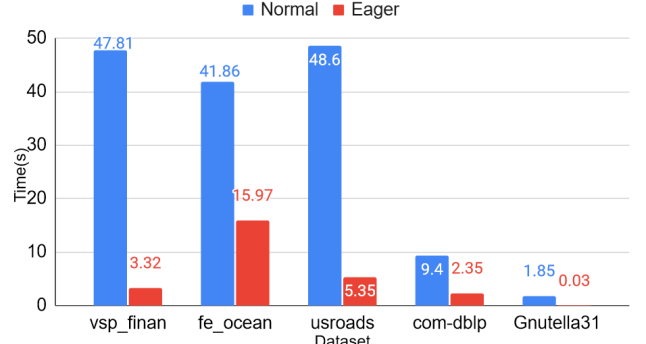
5.3 Evaluating Eager Buffer Management

We assess the efficacy of our Eager Buffer Management (EBM) mechanism by conducting a comparative analysis of runtime and memory usage with EBM enabled and disabled, using the *REACH* query on five distinct large graphs. Our results are presented in Table 1. This table is organized into four groups of columns. The first group specifies the datasets used in the experiment. The second group of columns provides insights into the iteration counts within the *REACH* query. This group is further divided into two sub-columns. “Total” denotes the total number of iterations, while “Tail” refers to the count of tail iterations, where the number of delta tuples generated in that iteration is less than 1% of the total tuples in the resulting Reachability relation. Notably, the absence of a tail iteration number for the “usroads” dataset is due to the fact that the number of delta tuples generated in every iteration of the *REACH* query on this dataset is less than 1%. The third group of columns displays the running time and speed-up ratio achieved by enabling Eager Buffer Management (EBM). It reveals that EBM accelerates all *REACH* queries, with greater benefits in longer iterations, especially long tail ones. The dataset “usroads” benefits the most, achieving a 3x acceleration.

In a separate analysis of detail buffer allocation time, represented in Figure 8, it’s evident that in datasets with extremely long tail relations like “vsp_finan” and “usroads”, buffer allocation speeds can approach up to 10x faster when EBM is active. Moreover, despite dataset “com-dblp” requiring more peak memory space and consequently larger buffer sizes, its buffer allocation time is actually lower than datasets like “vsp_finan”, which have smaller peak memory usage but longer tails. This observation suggests that buffer allocation costs are more closely linked to the frequency of buffer allocation operations than the total memory allocation size. Therefore, the additional memory space utilized by EBM does not incur a running time penalty, further substantiating EBM as a reasonable speed-up strategy.

From a memory perspective, Table 1 highlights that EBM introduces consistent memory overhead, maintaining peak memory

usage at around 1.35x the size of EBM-disabled runs. This demonstrates that EBM consistently adds memory overhead. However, the difference in the speedup-to-memory overhead ratio suggests that while EBM is a valuable strategy when buffer allocation is a bottleneck, it may not be as cost-effective in low iteration scenarios.

**Figure 8: Buffer allocation time with/without eager buffer management. (Normal is when Eager Buffer Management is disabled, Eager is when Eager Buffer Management is enabled)****Table 2: Reachability: GDLog vs. Soufflé (32 cores), GPUJoin, and cuDF on large graphs (out of memory marked as OOM).**

Dataset name	Reach edges	Time (s)			
		GDLog	Soufflé	GPUJoin	cuDF
com-dblp	1.91B	14.30	232.99	OOM	OOM
fe_ocean	1.67B	23.36	292.15	100.30	OOM
vsp_finan	910M	21.91	239.33	125.94	OOM
Gnutella31	884M	5.58	96.82	OOM	OOM
fe_body	156M	3.76	23.40	22.35	OOM
SF.cedge	80M	1.63	33.27	3.76	64.29

5.4 GDLog vs. preexisting GPU-based joins

In this section, we evaluate the performance of the *REACH* query using GDLog, GPUJoin, and cuDF, with souffle serving as the baseline for mature Datalog engines. The results are summarized in

Table 3: Same Generation (SG): GDLog vs. Soufflé (using 32 threads) and cuDF.

Dataset	SG size	Time (s)			
		GDLog	GDLog-HIP	Soufflé	cuDF
fe_body	408M	5.05	19.57	74.26	OOM
loc-Brightkite	92.3M	3.42	14.00	48.18	OOM
fe_sphere	205M	2.36	8.48	48.12	OOM
SF.cedges	382M	5.54	20.57	68.88	OOM
CA-HepTH	74M	2.79	5.92	20.12	21.24
ego-Facebook	15M	1.23	2.81	17.01	19.07

Table 4: Context-Sensitive Program Analysis (CSPA): GDLOG (NVIDIA H100) vs. Soufflé (EPYC 7543P on Polaris); input data from [5].

Dataset Name	Input Relation Size	Output Relation Size	Time (s)		Speedup
			GDLOG	Soufflé	
Httpd	Assign: 3.62e5 Dereference: 1.14e6	ValueFlow: 1.36e6 ValueAlias: 2.34e8 MemoryAlias: 8.89e7	1.33	49.48	37.2x
Linux	Assign: 1.98e6 Dereference: 7.50e6	ValueFlow: 5.50e6 ValueAlias: 2.23e7 MemoryAlias: 8.84e7	0.39	13.44	34.5x
PostgreSQL	Assign: 1.20e6 Dereference: 3.46e6	ValueFlow: 3.71e6 ValueAlias: 2.23e8 MemoryAlias: 8.84e7	1.27	57.82	44.9x

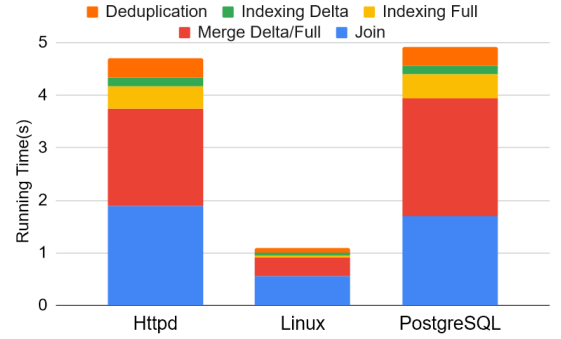
Table 2. The datasets used in the experiments represent a diverse array of sources, ensuring a comprehensive assessment that spans scientific communities, P2P networks, random graphs, and road networks. This dataset diversity helps mitigate any bias towards specific graph categories. The dataset sizes cover a broad spectrum of problem complexities.

GDLOG consistently emerges as the fastest engine in all test cases. Compared to GPUJoin, GDLOG exhibits a remarkable speedup, with ratios often exceeding 3x. In the “fe_body” testcase, the speedup ratio reaches an impressive 6x. Notably, GPUJoin, designed specifically for reachability queries, stores output relations in raw arrays. This specialization suggests that GDLOG could achieve even higher speedup ratios when the method of GPUJoin is applied to general deductive queries.

It is worth mentioning that GPUJoin also experienced two out-of-memory(OOM) errors during testing. This is attributed to its use of a pure open addressing hashmap for storing relations, necessitating a low hashtable loading factor to facilitate joins. In contrast, GDLOG’s HISA datastructure employs hash tables solely for accelerating range fetching for indexed relations, resulting in efficient access even with a higher load factor (0.8). Consequently, GDLOG outperforms GPUJoin in memory perspective and avoid OOM.

In contrast with GDLOG, cuDF’s implementation yields unsatisfactory results, even when compared to the baseline CPU-based approach. It’s complex code base makes it challenging to perform a detailed memory profile, but the presence of numerous out-of-memory errors during experiments points to memory-related performance issues. High memory usage significantly affects memory allocation and deallocation for buffer and tuple storage, contributing to slow running times.

We present another test case, SG, to further validate our previous discussions. The results are presented in Table 3. The SG query involves n-way joins and represents more general use cases. As GPUJoin does not support the SG query, we exclusively compare GDLOG with cuDF. Across all six test cases, cuDF encounters out-of-memory (OOM) errors four times, while the non-OOM test cases, although approaching the baseline performance of Soufflé, remain considerably slower than GDLOG. GDLOG consistently delivers stable performance, running nearly 7x faster than the baseline in most of cases, while not experiencing any OOM error.

**Figure 9: Running time detail analysis of CSPA query using GDLOG on various real world datasets**

5.5 Context-Sensitive Program Analysis

Program analysis is one of the most popular applications of Datalog, with several state-of-the-art program analysis engines (DOOP [8], cclzyer [6], and ddisasm [14]) implemented using Soufflé Datalog. To evaluate GDLOG’s ability to enable large-scale program analysis tasks, we have implemented the context-sensitive program analysis (CSPA), reproducing the experiments of Graspan [45].

The CSPA query, illustrated in Figure 10, includes two EDB relations: *assign*, recording static variable assignments in the target program, and *dereference*, tracking pointer dereferences; these relations are constructed by a preprocessing pass. The rules of the query forms an Andersen-style data flow analysis, encapsulated the results in the IDB relation *ValueFlow*, which delineates possible values of variables within specific calling contexts (encoded in the data via method cloning). The other two IDB relations, *MemoryAlias* and *ValueAlias*, record value and memory equivalences, respectively. The bottom five rules are used to initialize values in IDB relations; the mutually-recursive five upper rules declare the tuple population in the IDB rules. For instance, the first rule establishes the transitive nature of *ValueFlow*, while the second rule addresses the scenario where two values, flowing to the identical variable under the same context, are deemed equivalent (a situation indicative of value flow conflation and precision loss in context-sensitive point-to analysis).

Using input data from Graspan, we ran CSPA on three representative evaluation programs: Apache httpd, a significant subset of the Linux kernel (excluding many drivers), and PostgreSQL [45]. The results of our evaluation are presented in Table 4. The table details the size of input and output relations (second and third columns) and total query running time (fourth column). We validated that all relation sizes match that of Soufflé exactly. The large tuple sizes in both input and output make the query highly data-intensive, representing an attractive candidate for SIMD-based parallelization.

In sum, we see a roughly 35-45× speedup versus Soufflé under ideal conditions (souffle use 32 cores CPU with all optimizations, GDLOG running on H100 with maximum buffer size), demonstrating the significant performance advantage of GDLOG over the CPU-based Soufflé. To gain a deeper understanding of the running time characteristics of GDLOG and figuring out the running time bottleneck in the context of the CSPA query, we conducted a detailed breakdown of the CSPA’s running time, as shown in Figure 9. In this

$ValueFlow(x, y) \leftarrow ValueFlow(x, z), ValueFlow(z, y).$
 $ValueAlias(x, y) \leftarrow ValueFlow(z, x), ValueFlow(z, y).$
 $ValueFlow(x, y) \leftarrow assign(x, z), MemoryAlias(z, y).$
 $MemoryAlias(x, w) \leftarrow dereference(y, x), ValueAlias(y, z),$
 $dereference(z, w).$
 $ValueAlias(x, y) \leftarrow ValueFlow(z, x), MemoryAlias(z, w),$
 $ValueFlow(w, y).$
 $ValueFlow(y, x) \leftarrow assign(y, x).$
 $ValueFlow(x, x) \leftarrow assign(x, y).$
 $ValueFlow(x, x) \leftarrow assign(y, x).$
 $MemoryAlias(x, x) \leftarrow assign(y, x).$
 $MemoryAlias(x, x) \leftarrow assign(x, y).$

Figure 10: Datalog code for Context-Sensitive Program Analysis (CSPA). *assign* and *dereference* constitute EDB relations.

Table 5: Comparison of GDLOG and GDLOG-HIP running times across various GPU vendors and models

Query	Dataset	Time (s)			
		GDLOG (NVIDIA)		GDLOG-HIP (AMD)	
		H100	A100	MI250	MI50
SG	fe_body	5.05	8.61	19.57	41.99
	loc-BrightKite	3.42	6.79	14.00	30.05
	fe_sphere	2.36	4.64	8.48	19.426
CSPA	httpd	1.33	2.73	6.75	15.27
	linux	0.39	0.77	1.39	3.32
	postgresql	1.27	2.68	6.79	14.55

figure, “Deduplication” accounts for the time spent on removing tuples that already exist in the full relation. “Indexing Full/Delta” represents the time required for creating hash indexing within GDLOG’s hybrid data structure. “Merge Delta/Full” captures the time spent on inserting all tuples from the delta relation into the full relation using GPU merge. Finally, “Join” represents the actual time spent on performing join operations across all relations.

Figure 9 shows relative running time across different phases of GDLOG’s execution. We see the bottleneck in CSPA lies in the merge and join operations. As discussed previously, both of these components are highly data-intensive and can be significantly accelerated by leveraging GPU-based parallel sort and merge algorithms. Thus, we feel these results speak to the potential of GPU-based Datalog engines to scale to whole-program static analysis workloads.

Furthermore, a detailed examination of the CPU utilization ratio of Soufflé revealed that, despite utilizing all 32 CPU cores during computation, the efficiency of Soufflé was relatively low, with CPU ratios of 571%, 449%, and 683% for the httpd, Linux, and PostgreSQL datasets, respectively. This discrepancy is primarily due to the MIMD nature of multi-core-based engines, which struggle to effectively parallelize program-analysis queries with large volumes of data. In contrast, GPU-based Datalog engines like GDLOG have significant potential and can be harnessed for high-precision program-analysis queries on extensive open-source projects.

5.6 GDLOG Across GPUs

To compare GDLOG’s performance across a range of data center GPUs, we ran the SG and CSPA queries on the NVIDIA H100 and A100 and the AMD MI250 and MI50. We used our CUDA implementation for runs using NVIDIA GPUs and our HIP implementation for the AMD GPUs. The outcomes of these tests are presented in Table 5. The NVIDIA H100, the successor to the NVIDIA A100, boasts more Stream Multiprocessing (SM) units (114 vs. 108) and a doubled count of FP32 cores per SM unit (128 vs. 64), along with improved memory bandwidth (2TB/s vs. 1.5TB/s). Our results, presented in columns 4 and 5, highlight the benefits of these hardware enhancements. In every test, the H100 consistently delivers performance that is approximately twice as fast as the A100. This finding illustrates that GDLOG effectively harnesses the additional parallel computing resources offered by the latest GPU technology.

The last two columns illustrate GDLOG’s performance on AMD GPUs. The MI250, with 104 computational units (64 core per unit) akin to NVIDIA’s SMs, shows comparable performance to the NVIDIA A100 in CSPA tests but lags in SG tests requiring more memory, performing at about half the A100’s speed. This is partly due to HIP’s imperfect translation from CUDA to AMD’s ROCm and our eager buffer management technique’s reliance on the NVIDIA-specific RMM library, with manual memory pools on AMD not being as effective. The MI50, with half the capacity of the MI250, displays nearly half its performance, pointing towards the scalability of approach.

6 CONCLUSION AND FUTURE WORK

Modern CPU-based Datalog engines typically hit scalability walls around 8–16 threads, due to limited memory bandwidth and the overhead of locking data structures. The extreme parallel throughput offered by modern GPUs has made them an attractive candidate for the implementation of high-performance backends for Datalog that modern power program analyses and similar applications. However, current approaches to GPU Datalog are stymied by incomplete implementations, limited expressivity, and a lack of data structures specialized to the concerns of modern Datalog evaluation (iterated joins and semi-naïve evaluation).

We presented GDLOG, a new GPU-based system which achieves an order of magnitude performance improvement for large-scale deductive analytic queries. GDLOG is designed to compete with state-of-the-art CPU and GPU-based Datalog engines, and enables modern Datalog evaluation techniques, namely compilation to relational algebra, range-indexed iterated joins, and optimized n-way joins. GDLOG is powered by the HISA, a novel data structure we developed to support high-performance Datalog engines on the GPU. HISA combines the algorithmic necessities of modern Datalogs while effectively leveraging the massive SIMD parallelism available to modern GPUs. We then used HISA to build GDLOG by employing two novel design points: temporarily-materialized k -ary joins and eager buffer management. We have evaluated GDLOG extensively compared to both CPU and GPU-based state-of-the-art; our experimental results demonstrate improvements of up to 45× compared to an optimally-tuned CPU-based engine on context-sensitive points-to analysis of PostgreSQL.

We believe our work shows the potential for GPUs to bring orders-of-magnitude performance improvements to analytics applications in program analysis, graph mining, medical reasoning, and similar deduction-heavy domains; we are currently applying GDLog to large applications in several of these areas. As future work, we hope to explore multi-GPU joins, especially in the context of modern GPU interconnects and heterogeneous communication. We also hope to extend GDLog to enable incremental computations over large data streams on modern data center GPUs.

REFERENCES

- [1] AMD. 2024. *Hip documentation*. <https://rocm.docs.amd.com/projects/HIP/en/latest/>
- [2] Molham Aref, Balder ten Cate, Todd J Green, Benny Kimelfeld, Dan Olteanu, Emir Pasalic, Todd L Veldhuizen, and Geoffrey Washburn. 2015. Design and implementation of the LogicBlox system. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 1371–1382.
- [3] Argonne Leadership Computing Facility. 2024. *Aurora Supercomputer*. <https://www.alcf.anl.gov/aurora>
- [4] Argonne Leadership Computing Facility. 2024. *Polaris Supercomputer*. <https://www.alcf.anl.gov/polaris>
- [5] Muhammad A Awad, Saman Ashkiani, Rob Johnson, Martin Farach-Colton, and John D Owens. 2019. Engineering a high-performance gpu b-tree. In *Proceedings of the 24th symposium on principles and practice of parallel programming*. 145–157.
- [6] George Balatsouras and Yannis Smaragdakis. 2016. Structure-sensitive points-to analysis for C and C++. In *Static Analysis: 23rd International Symposium, SAS 2016, Edinburgh, UK, September 8–10, 2016, Proceedings 23*. Springer, 84–104.
- [7] Daniel Bartholomew. 2017. *Basics of Common Table Expressions*. Apress, Berkeley, CA, 3–9. https://doi.org/10.1007/978-1-4842-3120-3_1
- [8] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*. 243–262.
- [9] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (mar 2023), 1601–1614. <https://doi.org/10.14778/3587136.3587137>
- [10] Nvidia Corporation. 2019. *cuCollections*. <https://github.com/NVIDIA/cuCollections>
- [11] Chris J Date. 1989. *A Guide to the SQL Standard*. Addison-Wesley Longman Publishing Co., Inc.
- [12] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Trans. Math. Softw.* 38, 1, Article 1 (dec 2011), 25 pages. <https://doi.org/10.1145/2049662.2049663>
- [13] Zhiwei Fan, Jianqiao Zhu, Zuyu Zhang, Aws Albarghouthi, Paraschos Koutris, and Jignesh M. Patel. 2019. Scaling-up in-Memory Datalog Processing: Observations and Techniques. *Proc. VLDB Endow.* 12, 6 (feb 2019), 695–708. <https://doi.org/10.14778/3311880.3311886>
- [14] Antonio Flores-Montoya and Eric Schulte. 2020. Datalog disassembly. In *29th USENIX Security Symposium (USENIX Security 20)*. 1075–1092.
- [15] Oded Green. 2021. HashGraph—Scalable Hash Tables Using a Sparse Graph Data Structure. *ACM Trans. Parallel Comput.* 8, 2, Article 11 (jul 2021), 17 pages. <https://doi.org/10.1145/3460872>
- [16] Jiaqi Gu, Yugo H Watanabe, William A Mazza, Alexander Shkapsky, Mohan Yang, Ling Ding, and Carlo Zaniolo. 2019. RaSQL: Greater power and performance for big data analytics with recursive-aggregate-SQL on Spark. In *Proceedings of the 2019 International Conference on Management of Data*. 467–484.
- [17] Bingsheng He, Mian Lu, Ke Yang, Rui Fang, Naga K Govindaraju, Qiong Luo, and Pedro V Sander. 2009. Relational query coprocessing on graphics processors. *ACM Transactions on Database Systems (TODS)* 34, 4 (2009), 1–39.
- [18] Muhammad Idris, Martin Ugarte, and Stijn Vansummeren. 2017. The dynamic yannakakis algorithm: Compact and efficient query processing under updates. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1259–1274.
- [19] Muhammad Imran, Gábor E. Gévy, and Volker Markl. 2020. Distributed Graph Analytics with Datalog Queries in Flink. In *Software Foundations for Data Interoperability and Large Scale Graph Data Analytics*, Lu Qin, Wenjie Zhang, Ying Zhang, You Peng, Hiroyuki Kato, Wei Wang, and Chuan Xiao (Eds.). Springer International Publishing, Cham, 70–83.
- [20] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On synthesis of program analyzers. In *Computer Aided Verification: 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17–23, 2016, Proceedings, Part II 28*. Springer, 422–430.
- [21] Herbert Jordan, Pavle Subotić, David Zhao, and Bernhard Scholz. 2019. Brie: A specialized trie for concurrent datalog. In *Proceedings of the 10th International Workshop on Programming Models and Applications for Multicores and Manycores*. 31–40.
- [22] Herbert Jordan, Pavle Subotić, David Zhao, and Bernhard Scholz. 2019. A specialized B-tree for concurrent datalog evaluation. In *Proceedings of the 24th symposium on principles and practice of parallel programming*. 327–339.
- [23] Tomas Karnagel, René Müller, and Guy M Lohman. 2015. Optimizing GPU-accelerated Group-By and Aggregation. *ADMS@ VLDB 8* (2015), 20.
- [24] Sidharth Kumar and Thomas Gilray. 2020. Load-balancing parallel relational algebra. In *High Performance Computing: 35th International Conference, ISC High Performance 2020, Frankfurt/Main, Germany, June 22–25, 2020, Proceedings 35*. Springer, 288–308.
- [25] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [26] Feifei Li, Dihan Cheng, Marios Hadjieleftheriou, George Kollios, and Shang-Hua Teng. 2005. On trip planning queries in spatial databases. In *International symposium on spatial and temporal databases*. Springer, 273–290.
- [27] Carlos Alberto Martínez-Angeles, Inês Dutra, Vitor Santos Costa, and Jorge Buenabad-Chávez. 2013. A datalog engine for gpus. In *International Conference on Applications of Declarative Programming and Knowledge Management*. Springer, 152–168.
- [28] Frank McSherry, Derek Gordon Murray, Rebecca Isaacs, and Michael Isard. 2013. Differential Dataflow.. In *CIDR*.
- [29] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martin Abadi. 2013. Naiad: A Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (Farmington, Pennsylvania) (SOSP '13)*. Association for Computing Machinery, New York, NY, USA, 439–455. <https://doi.org/10.1145/2517349.2522738>
- [30] Yavor Nenov, Robert Piro, Boris Motik, Ian Horrocks, Zhe Wu, and Jay Banerjee. 2015. RDFox: A highly-scalable RDF store. In *The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11–15, 2015, Proceedings, Part II 14*. Springer, 3–20.
- [31] Oak Ridge National Laboratory. 2024. Aurora Supercomputer. <https://www.olcf.ornl.gov/frontier/>
- [32] Jun Rao and Kenneth A Ross. 1999. Cache Conscious Indexing for Decision-Support in Main Memory. In *Proceedings of the 25th International Conference on Very Large Data Bases*. 78–89.
- [33] RAPIDS. 2024. RMM: RAPIDS Memory Manager. <https://github.com/rapidsai/rmm>.
- [34] RAPIDS Development Team. 2021. cuDF: GPU DataFrame Library. <https://github.com/rapidsai/cudf>.
- [35] Daniel Ritter and Till Westmann. 2012. Business Network Reconstruction Using Datalog. In *Datalog in Academia and Industry*, Pablo Barceló and Reinhard Pichler (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 148–152.
- [36] Arash Sahebolamri, Langston Barrett, Scott Moore, and Kristopher Micinski. 2023. Bring Your Own Data Structures to Datalog. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 264 (oct 2023), 26 pages. <https://doi.org/10.1145/3622840>
- [37] Arash Sahebolamri, Thomas Gilray, and Kristopher Micinski. 2022. Seamless deductive inference via macros. In *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*. 77–88.
- [38] Jiwon Seo, Jongsoo Park, Jaeho Shin, and Monica S. Lam. 2013. Distributed Socialite: A Datalog-Based Language for Large-Scale Graph Analysis. *Proc. VLDB Endow.* 6, 14 (sep 2013), 1906–1917. <https://doi.org/10.14778/2556549.2556572>
- [39] Alexander Shkapsky, Mohan Yang, Matteo Interlandi, Hsuan Chiu, Tyson Condie, and Carlo Zaniolo. 2016. Big Data Analytics with Datalog Queries on Spark. In *Proceedings of the 2016 International Conference on Management of Data (San Francisco, California, USA) (SIGMOD '16)*. Association for Computing Machinery, New York, NY, USA, 1135–1149. <https://doi.org/10.1145/2882903.2915229>
- [40] Ahmedur Rahman Shovon. 2023. Public Github Repository of GPUJoin. <https://github.com/harp-lab/usenixATC23>.
- [41] Ahmedur Rahman Shovon, Landon Richard Dyken, Oded Green, Thomas Gilray, and Sidharth Kumar. 2022. Accelerating Datalog applications with cuDF. In *2022 IEEE/ACM Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 41–45.
- [42] Ahmedur Rahman Shovon, Thomas Gilray, Kristopher Micinski, and Sidharth Kumar. 2023. Towards iterative relational algebra on the {GPU}. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. 1009–1016.
- [43] Pavle Subotić, Herbert Jordan, Lijun Chang, Alan Fekete, and Bernhard Scholz. 2018. Automatic Index Selection for Large-Scale Datalog Computation. *Proc. VLDB Endow.* 12, 2 (oct 2018), 141–153. <https://doi.org/10.14778/3282495.3282500>
- [44] Todd L Veldhuizen. 2014. Leapfrog triejoin: A simple, worst-case optimal join algorithm. In *Proc. International Conference on Database Theory*.
- [45] Kai Wang, Aftab Hussain, Zhiqiang Zuo, Guoqing Harry Xu, and Ardalan Amiri Sani. 2017. Graspan: A Single-machine Disk-based Graph System for Interprocedural Static Analyses of Large-scale Systems Code. *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems* (2017). <https://api.semanticscholar.org/CorpusID:>

3655179

- [46] Haicheng Wu, Gregory Diamos, Tim Sheard, Molham Aref, Sean Baxter, Michael Garland, and Sudhakar Yalamanchili. 2014. Red fox: An execution environment for relational query processing on gpus. In *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*. 44–54.
- [47] Jiacheng Wu, Jin Wang, and Carlo Zaniolo. 2022. Optimizing parallel recursive datalog evaluation on multicore machines. In *Proceedings of the 2022 International Conference on Management of Data*. 1433–1446.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009